

Sutton and Barto — Chapter 3: Markov Decision Processes

The Agent-Environment Interface

The state contains the information used by the agent to choose an action.

state

S_t

reward

R_t

A real number that represents the consequence of the agent's action.

State and action are defined with more flexibility, but the reward is restricted to be a real number

$$R_{t+1} \in \mathcal{R} \subset \mathbb{R}$$

Agent

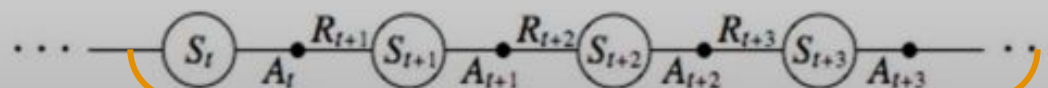
action

A_t

Action A_t taken at step t

Environment

A problem representation



Policy: the agent's behavior

The objective is to find a policy that maximizes the expected sum of rewards obtained by the agent.

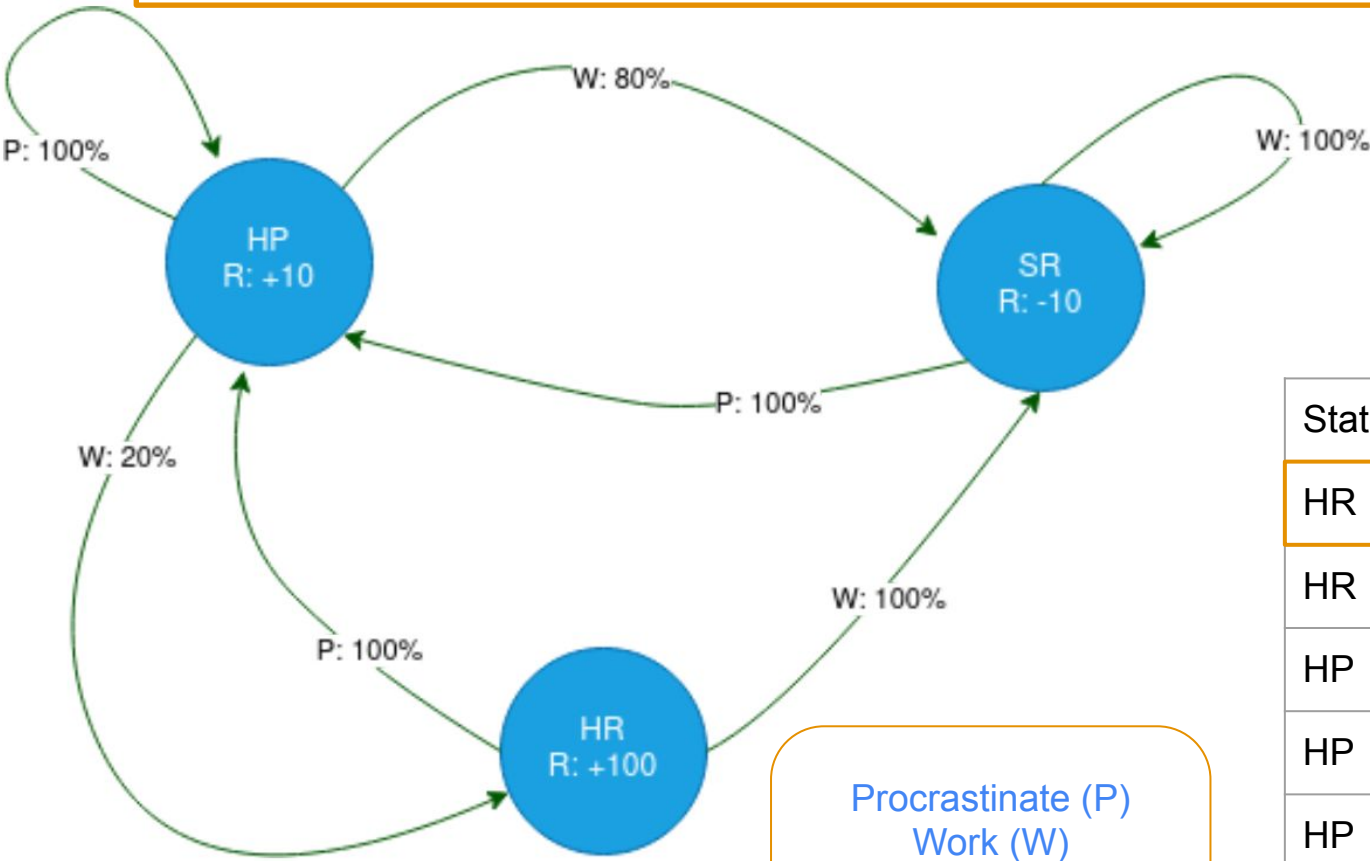
An episode is a sequence of time steps.

Finite Markov Decision Processes

In a finite MDP (FMDP), the sets of states, actions, and rewards all have a finite number of elements.

An MDP is defined by::

- Set of States S
- Initial State S_0
- Set of Actions A
- Transition Function $P(s' | s, a)$
- Reward Function $r(s, a, s')$
- Discount Factor γ



Procrastinate (P)
Work (W)
Happy and Rich (HR)
Happy and Poor (HP)
Sad and Rich (SR)

State	Action	Next State	Probability	Reward
HR	P	HP	100%	+10
HR	W	SR	100%	-10
HP	P	HP	100%	+10
HP	W	SR	80%	-10
HP	W	HR	20%	+100
SR	W	SR	100%	-10
SR	P	HP	100%	+10

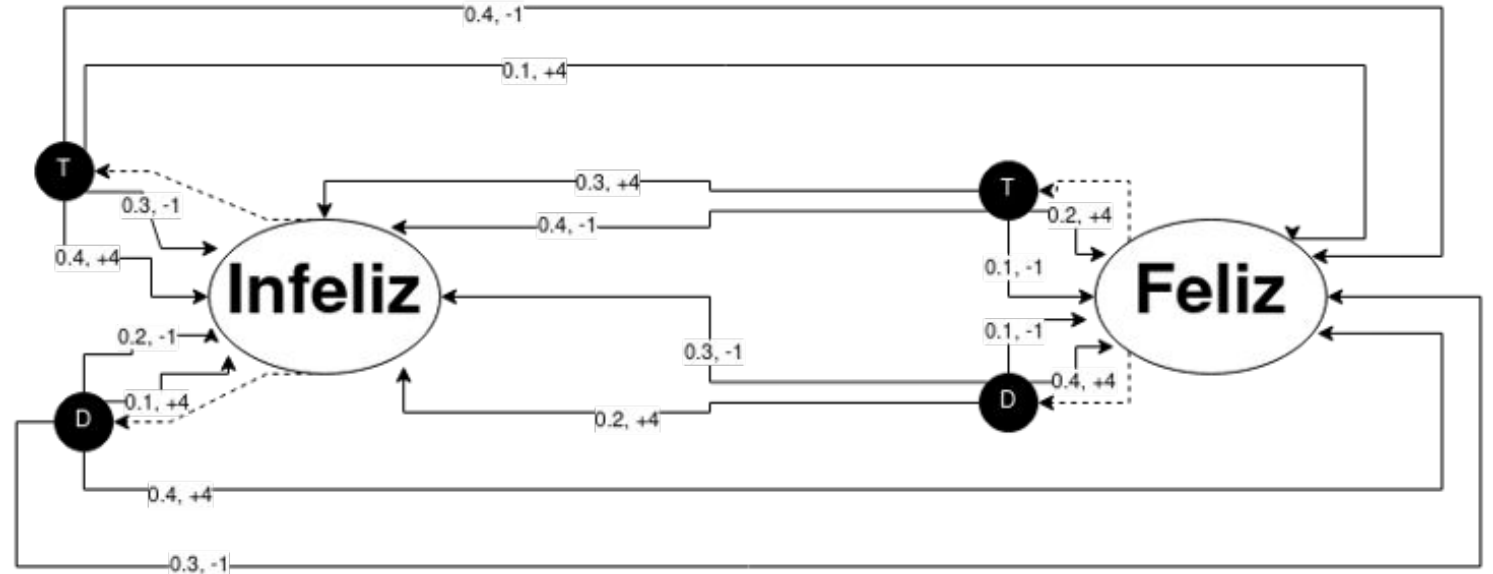
Finite Markov Decision Processes

An MDP can be described using the transition probabilities $p(s'|s,a)$ and the expected reward function $r(s,a,s')$. However, Sutton and Barto use a more general representation: $p(s',r|s,a)$

State 0: Feliz
State 1: Triste

Action 0: Trabalhar
Action 1: Descansar

Reward 0: -1
Reward 1: .4



For each state-action pair (s,a) , $p(s',r|s,a)$ defines a joint probability distribution over the next state s' and reward r .

s	a	s'	r	$p(s',r s,a)$
0	0	0	0	0.1
0	0	0	1	0.2
0	0	1	0	0.4
0	0	1	1	0.3
0	1	0	0	0.1
0	1	0	1	0.4
0	1	1	0	0.3
0	1	1	1	0.2

1	0	0	0	0.4
1	0	0	1	0.1
1	0	1	0	0.3
1	0	1	1	0.2
1	1	0	0	0.3
1	1	0	1	0.4
1	1	1	0	0.2
1	1	1	1	0.1

Probability Function for Finite Markov Decision Process (FMDP)

The probability function describes the environment dynamics by assigning probabilities to possible next states and rewards.

$$p(s', r | s, a) \doteq \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\}$$

$$p : \mathcal{S} \times \mathcal{R} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$$

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) = 1, \text{ for all } s \in \mathcal{S}, a \in \mathcal{A}(s)$$

Because it is considered a Finite MDP, the sum of the probability of all the possible achievable states and rewards must be 1.

It can be understood as the probability of reaching a next state s' and receiving reward r , given that the agent was in state s and took action a .

Computing other functions knowing the FMDP dynamics

From the four-argument dynamics function, p , one can compute anything else one might want to know about the environment, such as the *state-transition probabilities* (which we denote, with a slight abuse of notation, as a three-argument function $p : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$),

$$p(s' | s, a) \doteq \Pr\{S_t = s' \mid S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r | s, a). \quad (3.4)$$

We can also compute the expected rewards for state–action pairs as a two-argument function $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$:

$$r(s, a) \doteq \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r | s, a), \quad (3.5)$$

and the expected rewards for state–action–next-state triples as a three-argument function $r : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$,

$$r(s, a, s') \doteq \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a, S_t = s'] = \sum_{r \in \mathcal{R}} r \frac{p(s', r | s, a)}{p(s' | s, a)}. \quad (3.6)$$

Markov property: Introduction

Conditional Probability

1. $P(A|B) = P(A,B) / P(B)$
2. $P(A, B) = P(B) P(A|B)$

The probability of a event **A** and **B** happens is the probability of the event **B** multiply by the probability of **A** happens, given that **B** occurred before.

More Complex Case: A sequence of event A-B-C-D

$$\begin{aligned} P(A, B, C, D) &= \\ P(B, C, D) P(A | B, C, D) &= \\ P(C, D) P(B | C, D) P(A | B, C, D) &= \\ P(D) P(C | D) P(B | C, D) P(A | B, C, D) \end{aligned}$$

$$\begin{aligned} \text{Ex: } P(S_1, S_2, S_3) &= \\ P(S_1, S_2) P(S_3 | S_1, S_2) &= \\ P(S_1) P(S_2 | S_1) P(S_3 | S_1, S_2) \end{aligned}$$

Suppose we want to calculate the probability of observing the sequence of three states S_1, S_2, S_3 . This probability is equal to the probability of reaching S_1 , multiplied by the probability of reaching S_2 given that S_1 has already occurred, multiplied by the probability of reaching S_3 given that both S_1 and S_2 have already occurred.

First-order Markov property

$$P(A | B, C, D) = P(A | B)$$

$$\begin{aligned} P(A, B, C, D) &= \\ P(D) P(C | D) P(B | C) P(A | B) \end{aligned}$$

$$\begin{aligned} \text{Ex: } P(S_1, S_2, S_3) &= \\ P(S_1) P(S_2 | S_1) P(S_3 | S_2) \end{aligned}$$

Under the Markov property, the probability of each event in a sequence is conditioned only on the preceding event.

In an MDP, the next state and reward depend only on the current state and action, not on the full history.

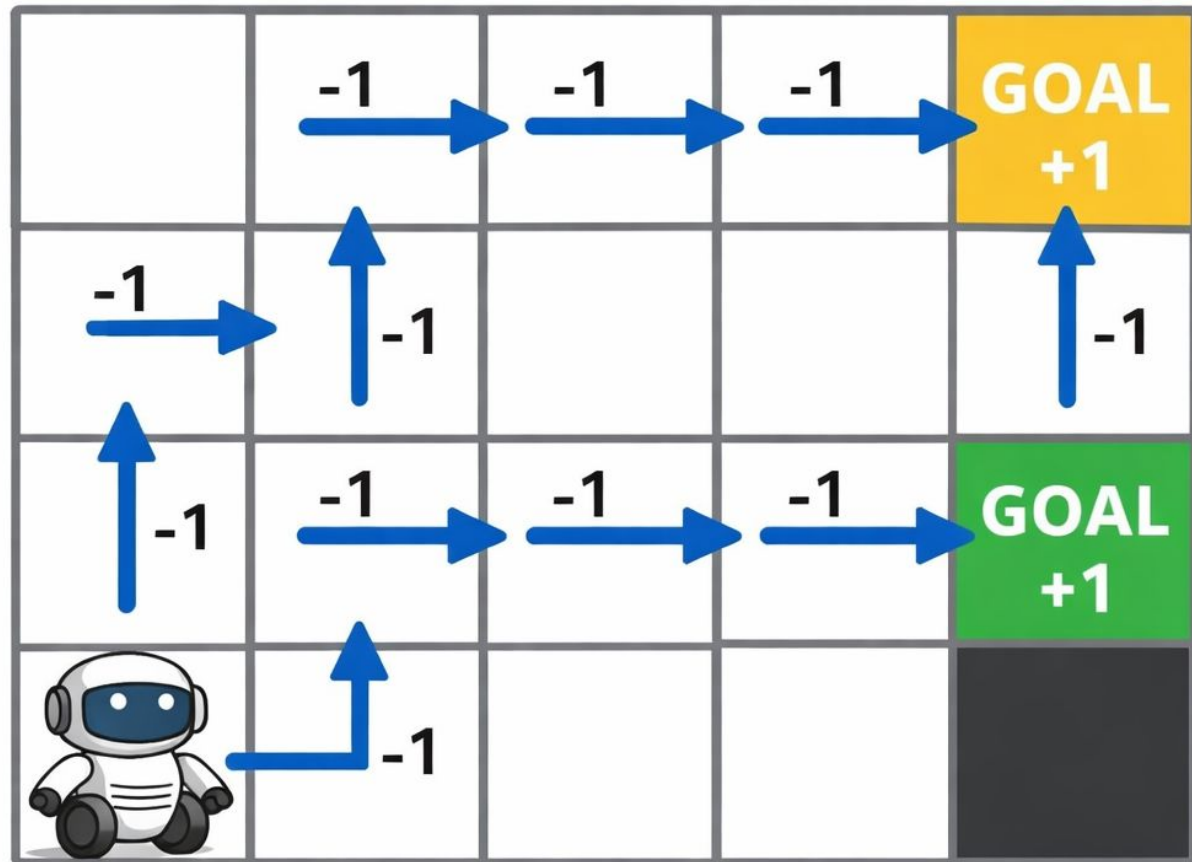
Goals and rewards

Let's say we have the following problem:

A robot moves in a grid world and aims to reach a goal state in as few steps as possible. It can move only up, down, left, or right. How can we model this problem?

First, we define the objective and the reward function. The robot receives +1 when it reaches the goal state and -1 at each non-terminal step, encouraging it to find the shortest path.

*"The use of a **reward** signal to formalize the idea of a **goal** is one of the most distinctive features of reinforcement learning."*



Returns and Episodes

Now, we must define if we want to interpret this problem as continuing or episodic.

In an episodic task, interaction is divided into episodes. Each episode ends when a terminal state is reached or when a maximum time limit is imposed.

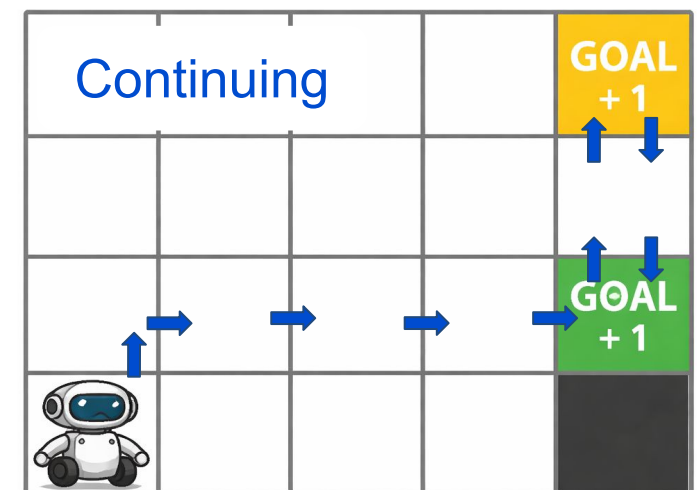
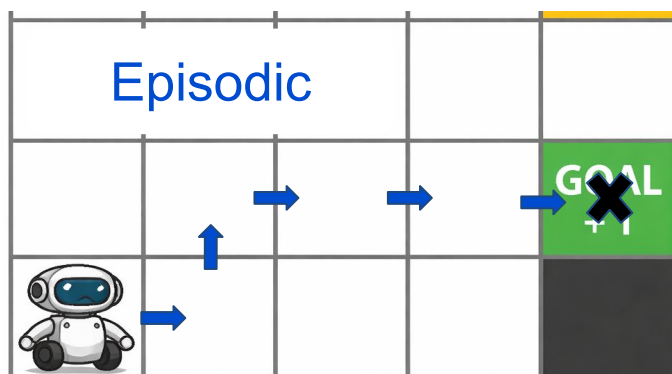
In this case, the objective of the agent is to maximize the sum of the rewards obtained in one episode. It is also called Return (G).

In a continuing task, there is no natural terminal state. A discount factor γ is commonly used to make future rewards less influential than immediate rewards. The Return now has to consider γ .

Expected (long-term) return G is what we want to maximize, instead of the (immediate) reward R

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$



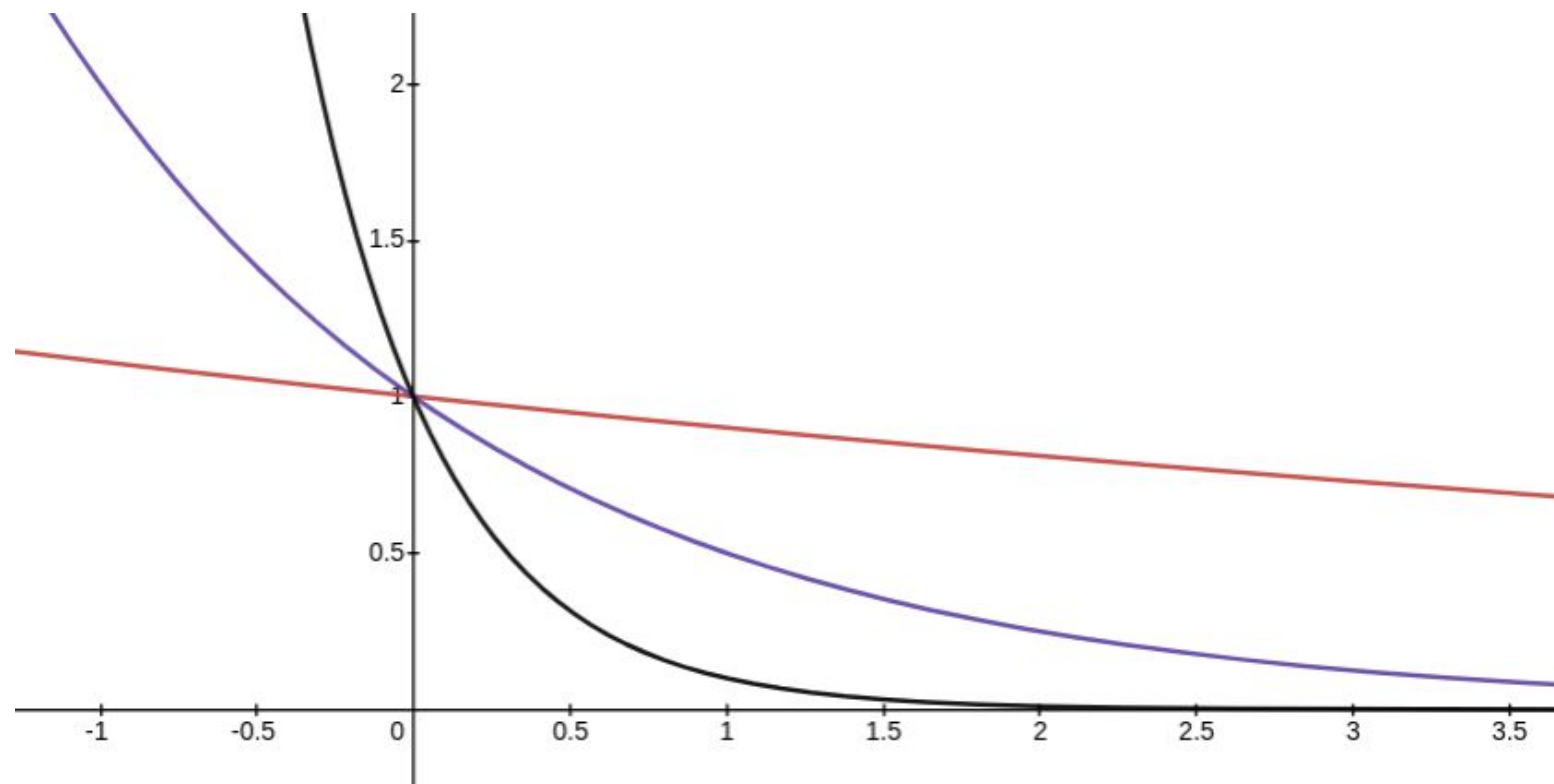
Returns and Episodes

What if gamma is almost 1 ?

If $\gamma < 1$

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

1		$y = 0.9^x$
2		$y = 0.5^x$
3		$y = 0.1^x$

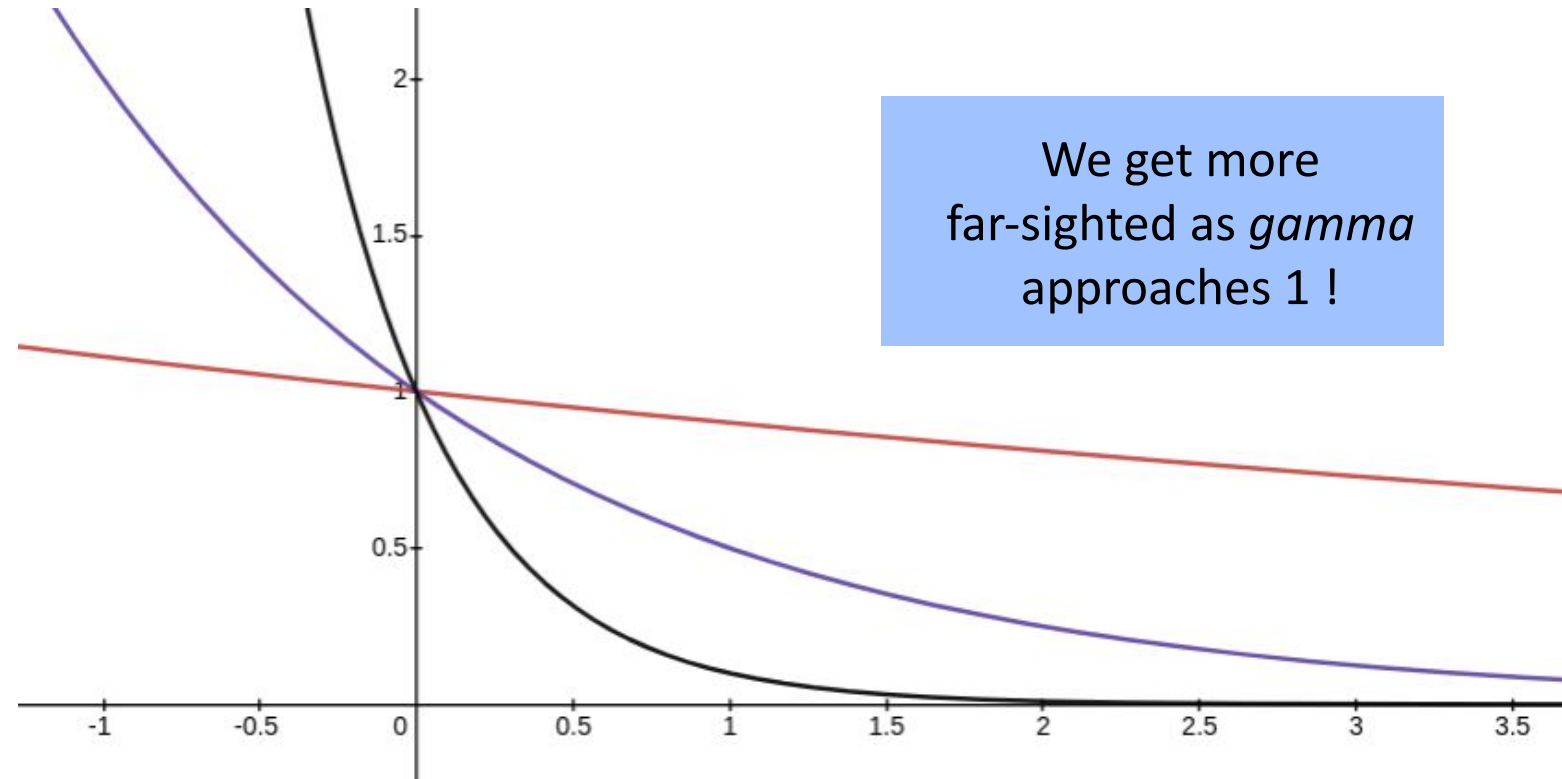


Returns and Episodes

If $\gamma < 1$

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

1		$y = 0.9^x$
2		$y = 0.5^x$
3		$y = 0.1^x$



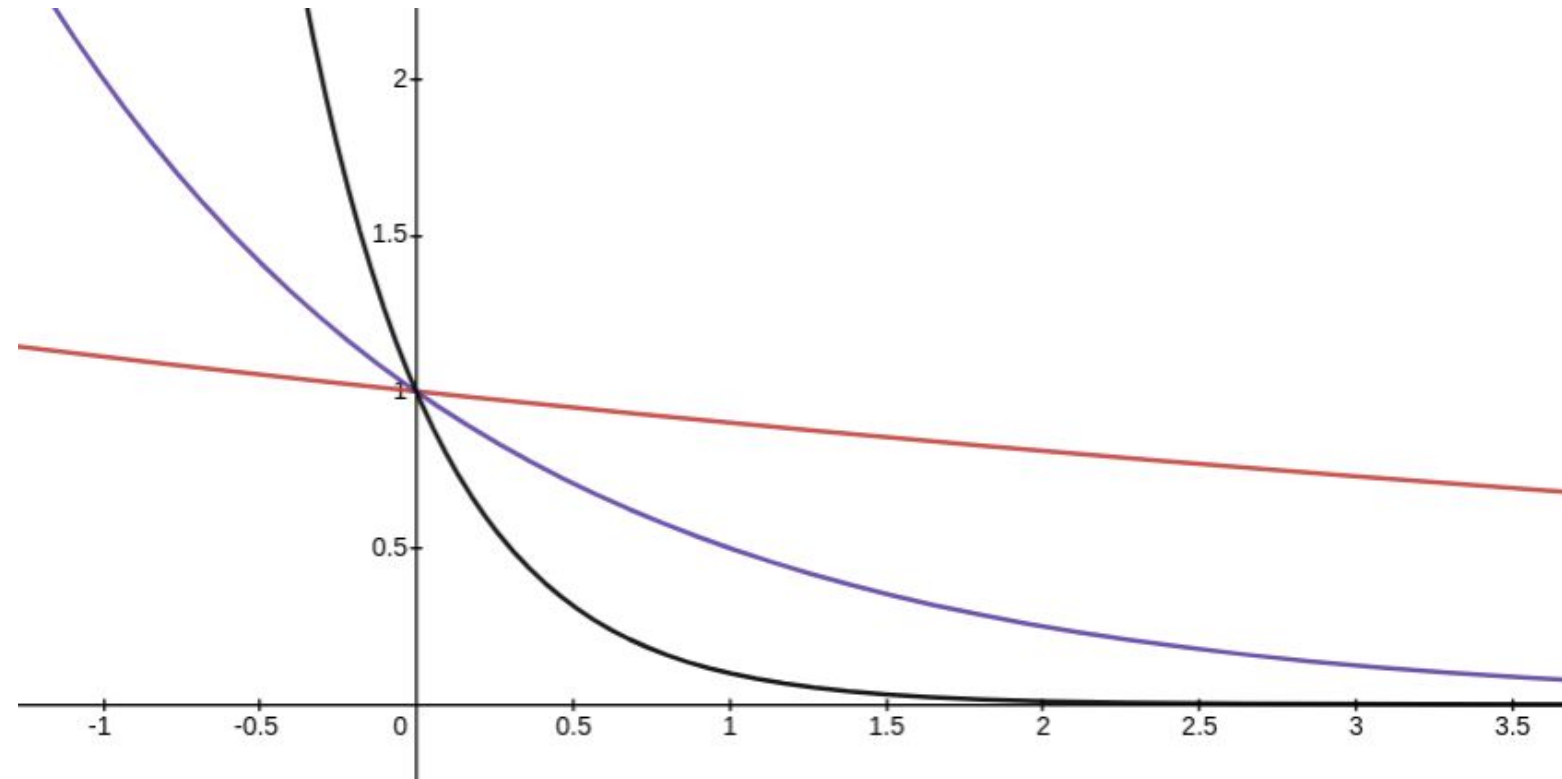
Returns and Episodes

If $\gamma < 1$

What if $\gamma = 0$?

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

1		$y = 0.9^x$
2		$y = 0.5^x$
3		$y = 0.1^x$



Returns and Episodes

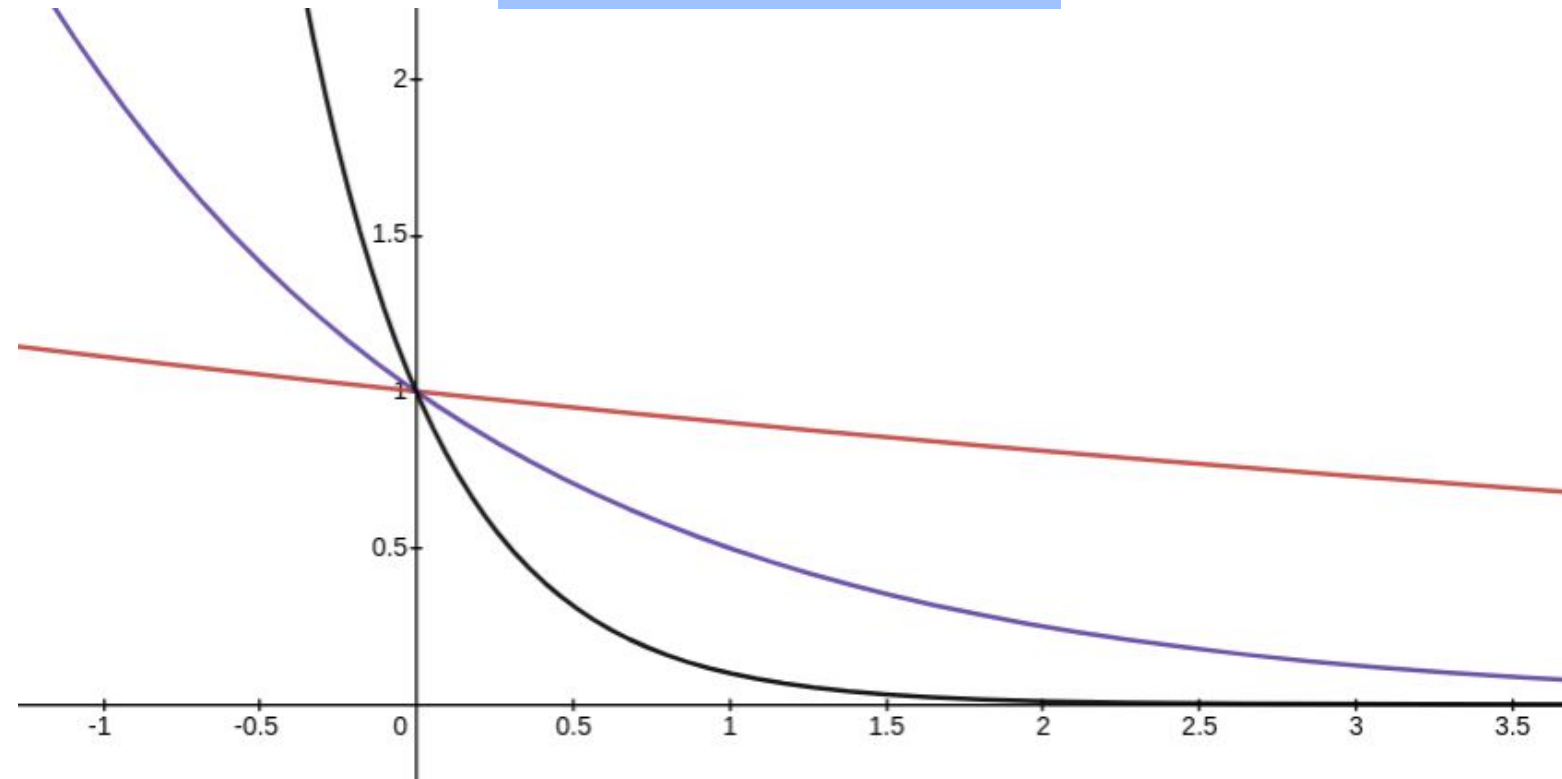
If $\gamma < 1$

What if $\gamma = 0$?

$$G_t \doteq R_{t+1}$$

Then we only consider
the next reward

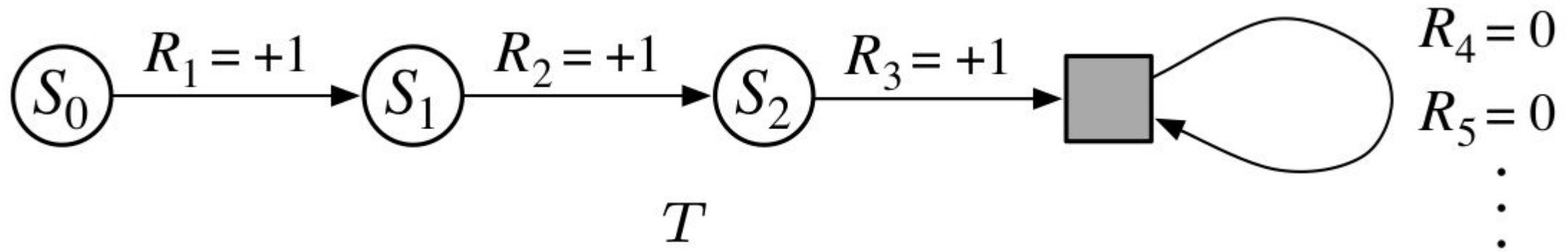
1		$y = 0.9^x$
2		$y = 0.5^x$
3		$y = 0.1^x$



Unified Notation for Episodic and Continuing Tasks

Notation: consider episode termination to be the entering of a special absorbing state that transitions only to itself and that generates only rewards of zero. This allows us to use the same equation:

$$G_t \doteq \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$



$$G_t \doteq \sum_{k=t+1}^T \gamma^{k-t-1} R_k$$

Terminal
Time Step

$$\boxed{T} = \infty \text{ or } \gamma = 1 \text{ (but not both)}$$

Iteratively calculating return G

$$\begin{aligned} G_t &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \\ &= R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

The return for the final Step is equal to 0.

$$G_T = 0$$

The return at a given time step is the immediate reward plus the discounted return from the next time step.

Unless the person is religious, there will be only rewards equal to zero after the last day (time T) of life on Earth

Exercise 3.8

Suppose $\gamma = 0.5$ and the following sequence of rewards is received $R_1 = -1$, $R_2 = 2$, $R_3 = 6$, $R_4 = 3$, and $R_5 = 2$, with $T = 5$. What are G_0 , $G_1, \dots, G_5$? Hint: Work backwards.

Calculate G at time $t=0$, G_0 :

$$G_0 = 0.5 \cdot 2 + 0.25 \cdot 6 + 0.125 \cdot 3 + 0.0625 \cdot 2 = 2$$

Calculate G at time $t=1$, G_1 :

$$G_1 = 2 + 0.5 \cdot 6 + 0.25 \cdot 3 + 0.125 \cdot 2 = 6$$

Calculate recursively backwards

$$G_5 = 0$$

$$G_4 = 2 + 0.5G_5 = 2$$

$$G_3 = 3 + 0.5G_4 = 4$$

$$G_2 = 6 + 0.5G_3 = 8$$

$$G_1 = 2 + 0.5G_2 = 6$$

$$G_0 = -1 + 0.5G_1 = 2$$

```
1 import numpy as np
2 r=[-1, 2, 6, 3, 2]
3 gamma=0.5 # discount rate
4 # Calculate G for time t=0
5 n=[0, 1, 2, 3, 4]
6 print('G0=', np.sum(r * np.power(gamma,n)))
7 print('G1=', np.sum(r[1:] * np.power(gamma,n[:-1])))
8 # Calculate G recursively and backwards for all time t
9 Gt=0 # initialize G_T = 0
10 for t in np.arange(4,-1,-1):
11     Gtp1=r[t]+gamma*Gt # recursive equation
12     print('G' + str(t) + '=' + str(Gtp1))
13     Gt=Gtp1 # update for next iteration
```

Loop output:

$G_4=2.0$

$G_3=4.0$

$G_2=8.0$

$G_1=6.0$

$G_0=2.0$

Modeling the Problem as FMDP

For the previous problem, let's define the RL elements:

State: The robot's position

$S = \{(0,0), (0,1)\dots, (4,3)\}$

Action: The robot's movement

$A = \{\text{Up, Down, Left, Right}\}$

Reward: Achieve the goal or not

$R = \{-1, +1\}$

Remember that it is finite because all the sets are finite.

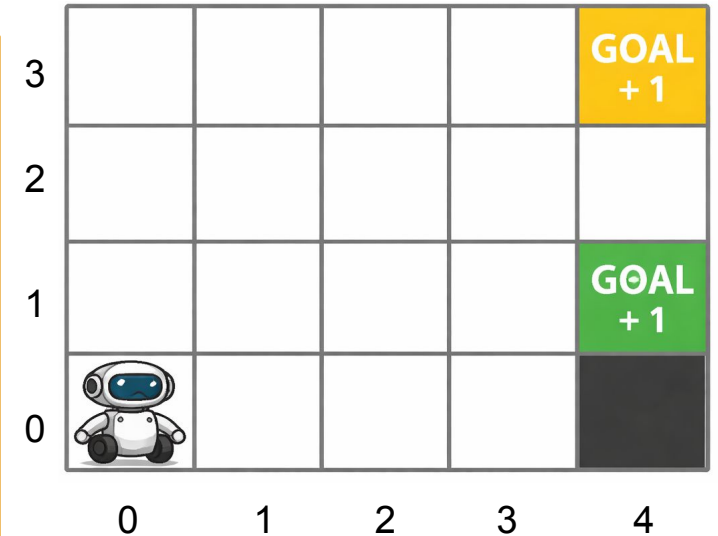
How do we get the probabilities?

First, let's assume: $s = (0, 0)$ and $a = (\text{Up})$. The probability for it reaches the state $s' = (0, 1)$ or $(4,3)$ is zero, Because when it go up, it goes to $s' = (1, 0)$. So the probability $P((0,1) / (\text{Up}), (0, 0)) = 1$.

Now for the most general representation, we also have to consider the reward.

$P((0,1), +1 / (\text{Up}), (0, 0)) = 0$, because $(1, 0)$ is not the goal.

$P((0,1), -1 | (\text{Up}), (0, 0)) = 1$



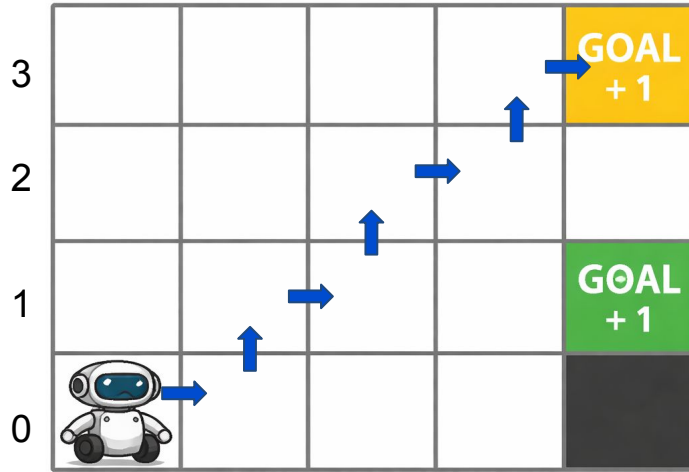
$P(s' / a, s)$

$R(s' / a, s)$

$P(s', r / a, s)$

Modeling the Problem: Policy

Consider two episodic situations:



which one, the robot make better decisions?

In the First, the return is:
 $G = -1-1-1-1-1-1+1 = -5$

In the Second, the return is:
 $G = -1-1-1-1+1 = -3$

So we can notice that the return depends on the agent's behavior.

This behavior is called **Policy**
 $\Pi(a / s)$

It is the probability of take an action given a state.

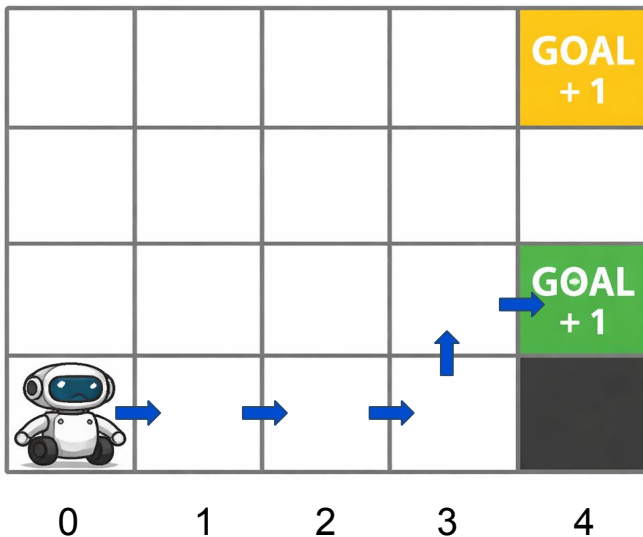
How to get a policy?

In the First case, what is the probability of the robot go to left when it is at state (0, 0)?

$$\Pi_1(\text{Left} / (0,0)) = 1$$

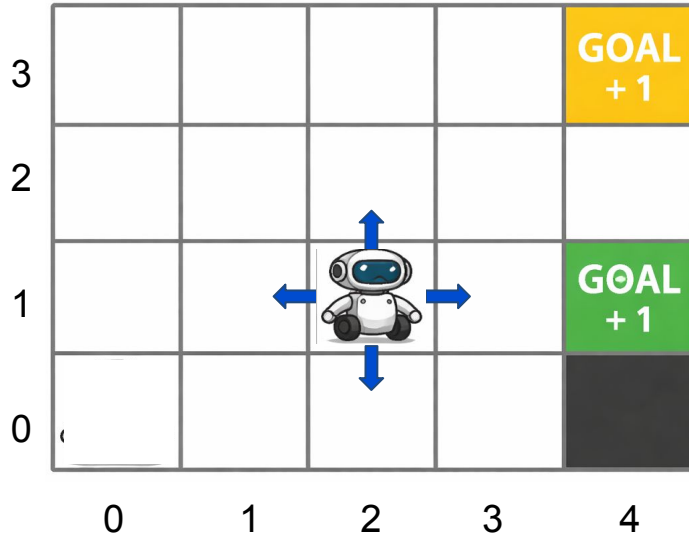
In the second case, for the state (0, 1), the probability to go Up is 0, but to the left is 1.

In both cases, this policy is called **Deterministic**.



Modeling the Problem: Policy

A third Case:



In this situation, the behavior of the robot says that it can take any of the possible actions with equal probability for the state (2, 1). So:

$$\Pi_3(\text{Left} / (2, 1)) = 0.25$$

$$\Pi_3(\text{Right} / (2, 1)) = 0.25$$

$$\Pi_3(\text{Up} / (2, 1)) = 0.25$$

$$\Pi_3(\text{Down} / (2, 1)) = 0.25$$

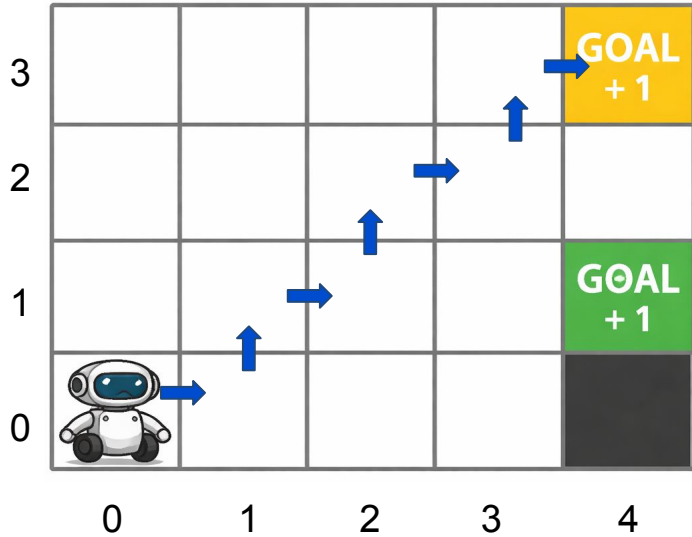
Remember that for a given state, the sum of the probability of all actions must be equal to 1.

Observations:

In these three cases, it can be observed that the policy is not explicitly defined for all states. In practice, the agent will be able to take an action for all states, even if it is a uniform distribution over all actions.

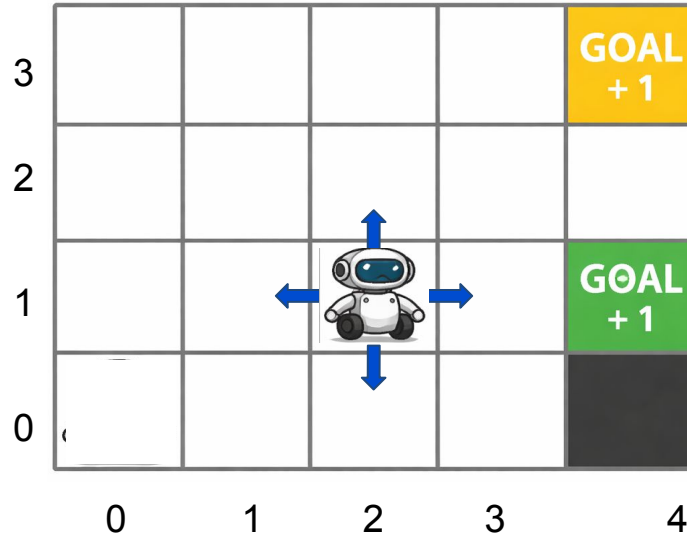
In this case, the policy is **stochastic**.

Modeling the Problem: Return



For a deterministic policy, the return can be computed directly along the trajectory:

$$G_t = R_{t+1} + \gamma G_{t+1}$$



However, this is not as simple in the case of a stochastic policy.

From a given state, the policy may choose different actions with different probabilities, and even after, the environment may lead to different next states and rewards.

Now, we must consider an expected return given a state.

$$\mathbb{E}[G_t | S_t = s]$$

This can be rewritten as:

$$\mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s]$$

by the linearity of the expectation:

$$\mathbb{E}[R_{t+1} | S_t = s] + \gamma \mathbb{E}[G_{t+1} | S_t = s]$$

Modeling the Problem: Return

However, there are two main sources of randomness:

1. The policy $\Pi(\mathbf{a} / \mathbf{s})$
2. The transition function $P(\mathbf{s}', \mathbf{r} \mid \mathbf{s}, \mathbf{a})$

Let us first interpret what the previous equation means:

The first term corresponds to the expected immediate **rewards**, considering the **actions** selected by the policy and the possible **transitions** generated by the environment

$$\mathbb{E}[R_{t+1} | S_t = s] + \gamma \mathbb{E}[G_{t+1} | S_t = s]$$

The second term corresponds to the expected return from the next time step.

It also will consider both the aspects of **action** taken and the **transition** observed by the agent. This future return is weighted by the **discount factor** γ .

The expected return of a state is the reward we expect to receive now plus the discounted return we expect to receive from the next state.

Modeling the Problem: Return

Now, we can expand the previous equation to:

We are running over all possible actions

$$\sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) R(s',a,s) + \gamma \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) \mathbb{E}[G_{t+1}|S_t = s, S_{t+1} = s']$$

We are running over all possible next states and rewards

$$\sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) R(s',a,s) + \gamma \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) \mathbb{E}[G_{t+1}|S_{t+1} = s']$$

Using the Markov property, this becomes:

Then, it can be simplified to:

$$\sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [R(s',a,s) + \gamma \mathbb{E}[G_{t+1}|S_{t+1} = s']]$$

Modeling the Problem: Value Function

The expected return from a given state under policy π is called the state-value function $V_\pi(\mathbf{S})$

$$V_\pi(S) = \mathbb{E}[G_t | S_t = s]$$

That can be calculated with the previous equation:

$$V_\pi(S) = \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [R(s, a, s) + \gamma V_\pi(S')]$$

This is also called as the **Bellman's Equation for the Value Function**.

The expected return after taking action a in state s , following a policy π , is called the action-value function. $Q_\pi(\mathbf{S}, \mathbf{A})$

$$Q_\pi(S, A) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

For both value functions there are optimal values obtained under an optimal policy and they are represented for *

$V_*(\mathbf{S})$: Optimal VF
 $Q_*(\mathbf{S}, \mathbf{A})$: Optimal AVF
 $\pi_*(\mathbf{A}|\mathbf{S})$: Optimal Policy

Example: Value Function

Let's consider the following scenario: an FMDP with just 2 states and two actions. γ is 0.9 and the policy is given by:

$$\begin{aligned}\pi(a_1 | s_1) &= 0.5 & \pi(a_2 | s_1) &= 0.5 \\ \pi(a_1 | s_2) &= 0.8 & \pi(a_2 | s_2) &= 0.2\end{aligned}$$

The rewards and transition probabilities are given by $R(S, A, S')$ and $P(S' | A, S)$, respectively:

S	A	S'	R	P
s1	a1	s1	0	1
s1	a2	s2	1	1
s2	a1	s1	2	1
s2	a2	s2	0	1

Remember that the Value Function is given by:
$$V_{\pi}(S) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [R(s,a,s) + \gamma V_{\pi}(S')]$$

$$\begin{aligned}\text{For } s1: \quad V_{\pi}(s1) &= 0.5[0 + 0.9V_{\pi}(s1)] + 0.5[1 + 0.9V_{\pi}(s2)] \Rightarrow V_{\pi}(s1) = 0.45V_{\pi}(s1) + 0.5 + 0.45V_{\pi}(s2) \\ \text{For } s2: \quad V_{\pi}(s2) &= 0.8[2 + 0.9V_{\pi}(s1)] + 0.2[0 + 0.9V_{\pi}(s2)] \Rightarrow V_{\pi}(s2) = 1.6 + 0.72V_{\pi}(s1) + 0.18V_{\pi}(s2)\end{aligned}$$

Example: Value Function

Lets consider the following scenario: A FMDP with just 2 states and two actions. γ is 0.9 and the policy is given by:

$$\begin{aligned}\pi(a_1 | s_1) &= 0.5 & \pi(a_2 | s_1) &= 0.5 \\ \pi(a_1 | s_2) &= 0.8 & \pi(a_2 | s_2) &= 0.2\end{aligned}$$

The rewards and transition probabilities will be given by $R(S, A, S')$ and $P(S' | A, S)$, respectively:

S	A	S'	R	P
s1	a1	s1	0	1
s1	a2	s2	1	1
s2	a1	s1	2	1
s2	a2	s2	0	1

let's say that $V_{\pi}(s1) = a$, and $V_{\pi}(s2) = b$ then:

$$\begin{aligned}a &= 0.45a + 0.5 + 0.45b \Rightarrow 0.55a - 0.45b = 0.5 \\ b &= 1.6 + 0.72a + 0.18b \Rightarrow -0.72a + 0.82b = 1.6\end{aligned}$$

It is a linear equation system

Then, solving the system:

$$V_{\pi}(s1) = a = 7.04 \text{ and } V_{\pi}(s2) = b = 8.61$$

Optimal Values and policy

For both value functions there are optimal values obtained when using a optimal policy and they are represented for *. The optimal policy is the one that will give the higher rewards and (consequently) returns possible.

$V_*(S)$: Optimal VF

$Q_*(S, A)$: Optimal AVF

$\pi_*(A|S)$: Optimal Policy

So, we can adapt both the bellman equation to calculate the optimal value functions i.e. **Optimal Bellman's Equation.**

$$V_*(S) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V_*(S')]$$

$$Q_*(S, A) = \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} Q_*(S', A')]$$

To pass one to another:

$$Q_*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma V_*(s')]$$

$$V_*(S) = \operatorname{argmax}_A Q_*(S, A)$$

Optimal Values and policy

Let's say we already have the $Q_*(S, A)$, how can we get the optimal policy from this?

Consider the following optimal action value function given by a table.

	s1	S2
a1	1	2
a2	-1	-2

We can read this table as: The value of the action 1 in the state 1 is 1...

The optimal policy will just the action with high AVF for a give state, so:

$$\pi_*(a1|s1) = 1 \quad \pi_*(a2|s1) = 0$$

$$\pi_*(a1|s2) = 1 \quad \pi_*(a2|s2) = 0$$

$$\pi_*(S) = \operatorname{argmax}_a Q_*(S,A)$$

The objective os the RL training is for the agent achieves an optimal policy (or close to it). For that, some strategies first needs to estimate the AVF using Optimal Bellman's Equation.

Greedy policy

Example 3.5: Gridworld Figure 3.2 (left) shows a rectangular gridworld representation of a simple finite MDP. The cells of the grid correspond to the states of the environment. At each cell, four actions are possible: **north**, **south**, **east**, and **west**, which deterministically cause the agent to move one cell in the respective direction on the grid. Actions that would take the agent off the grid leave its location unchanged, but also result in a reward of -1 . Other actions result in a reward of 0 , except those that move the agent out of the special states **A** and **B**. From state **A**, all four actions yield a reward of $+10$ and take the agent to **A'**. From state **B**, all actions yield a reward of $+5$ and take the agent to **B'**.

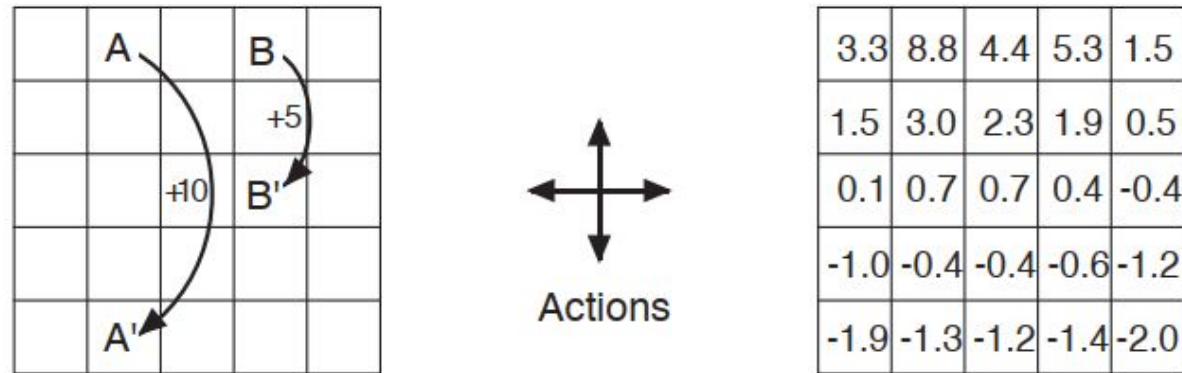


Figure 3.2: Gridworld example: exceptional reward dynamics (left) and state-value function for the equiprobable random policy (right).

Optimal policy

Example 3.8: Solving the Gridworld Suppose we solve the Bellman equation for v_* for the simple grid task introduced in Example 3.5 and shown again in Figure 3.5 (left). Recall that state A is followed by a reward of +10 and transition to state A', while state B is followed by a reward of +5 and transition to state B'. Figure 3.5 (middle) shows the optimal value function, and Figure 3.5 (right) shows the corresponding optimal policies. Where there are multiple arrows in a cell, all of the corresponding actions are optimal.

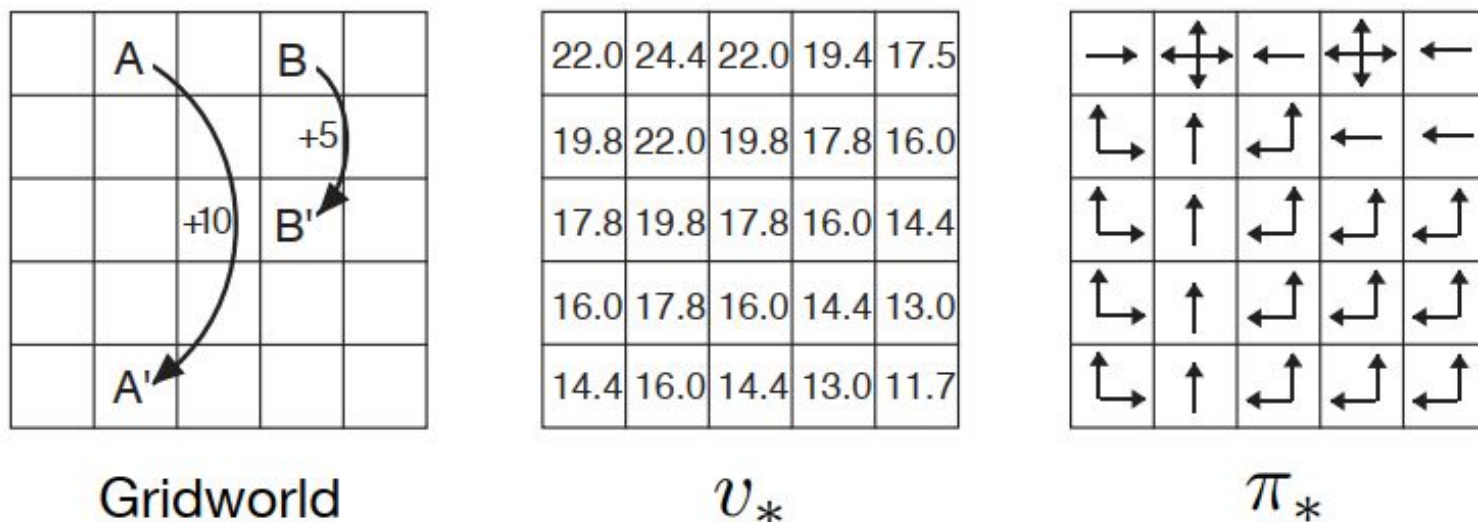
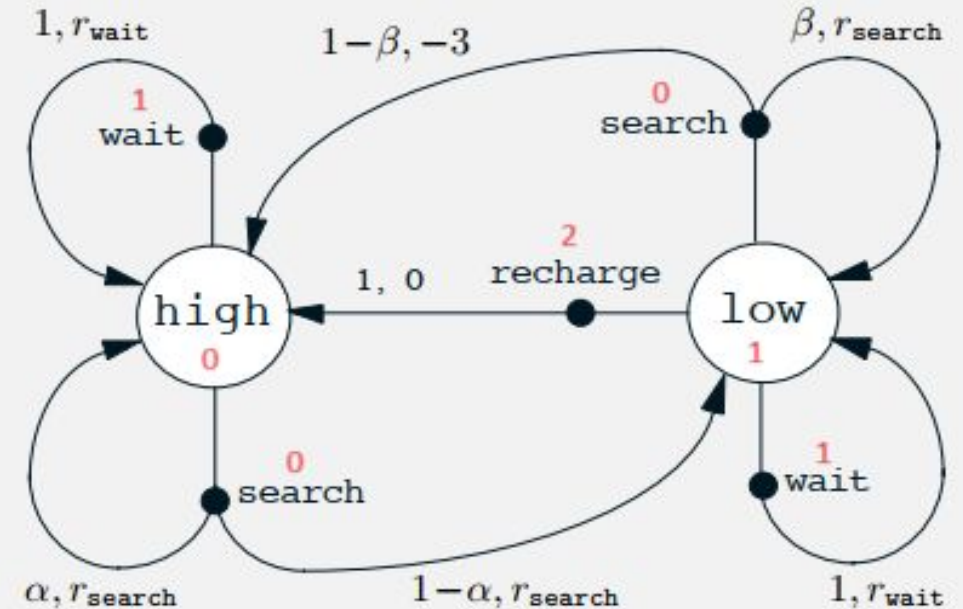


Figure 3.5: Optimal solutions to the gridworld example.

How to solve the optimal bellman equation? Mathematical Way

Let's consider the recycle robot problem from the book

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	$-$
low	wait	high	0	$-$
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	$-$



Recycle Robot Problem

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	-

$$V_*(S) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V_*(S')]$$

$$\begin{aligned}
 v_*(\mathbf{h}) &= \max \left\{ \begin{array}{l} p(\mathbf{h} | \mathbf{h}, \mathbf{s}) [r(\mathbf{h}, \mathbf{s}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{l} | \mathbf{h}, \mathbf{s}) [r(\mathbf{h}, \mathbf{s}, \mathbf{l}) + \gamma v_*(\mathbf{l})], \\ p(\mathbf{h} | \mathbf{h}, \mathbf{w}) [r(\mathbf{h}, \mathbf{w}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{l} | \mathbf{h}, \mathbf{w}) [r(\mathbf{h}, \mathbf{w}, \mathbf{l}) + \gamma v_*(\mathbf{l})] \end{array} \right\} \\
 &= \max \left\{ \begin{array}{l} \alpha [r_{\mathbf{s}} + \gamma v_*(\mathbf{h})] + (1 - \alpha) [r_{\mathbf{s}} + \gamma v_*(\mathbf{l})], \\ 1 [r_{\mathbf{w}} + \gamma v_*(\mathbf{h})] + 0 [r_{\mathbf{w}} + \gamma v_*(\mathbf{l})] \end{array} \right\} \\
 &= \max \left\{ \begin{array}{l} r_{\mathbf{s}} + \gamma [\alpha v_*(\mathbf{h}) + (1 - \alpha) v_*(\mathbf{l})], \\ r_{\mathbf{w}} + \gamma v_*(\mathbf{h}) \end{array} \right\}.
 \end{aligned}$$

Recycle Robot Problem

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	-

$$\begin{aligned}
 v_*(\mathbf{h}) &= \max \left\{ \begin{array}{l} p(\mathbf{h} | \mathbf{h}, \mathbf{s})[r(\mathbf{h}, \mathbf{s}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{1} | \mathbf{h}, \mathbf{s})[r(\mathbf{h}, \mathbf{s}, \mathbf{1}) + \gamma v_*(\mathbf{1})], \\ p(\mathbf{h} | \mathbf{h}, \mathbf{w})[r(\mathbf{h}, \mathbf{w}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{1} | \mathbf{h}, \mathbf{w})[r(\mathbf{h}, \mathbf{w}, \mathbf{1}) + \gamma v_*(\mathbf{1})] \end{array} \right\} \\
 &= \max \left\{ \begin{array}{l} \alpha[r_s + \gamma v_*(\mathbf{h})] + (1 - \alpha)[r_s + \gamma v_*(\mathbf{1})], \\ 1[r_w + \gamma v_*(\mathbf{h})] + 0[r_w + \gamma v_*(\mathbf{1})] \end{array} \right\} \\
 &= \max \left\{ \begin{array}{l} r_s + \gamma[\alpha v_*(\mathbf{h}) + (1 - \alpha)v_*(\mathbf{1})], \\ r_w + \gamma v_*(\mathbf{h}) \end{array} \right\}.
 \end{aligned}$$

Recycle Robot Problem

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	-

$$\begin{aligned}
 v_*(\mathbf{h}) &= \max \left\{ \begin{array}{l} p(\mathbf{h} | \mathbf{h}, \mathbf{s})[r(\mathbf{h}, \mathbf{s}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{l} | \mathbf{h}, \mathbf{s})[r(\mathbf{h}, \mathbf{s}, \mathbf{l}) + \gamma v_*(\mathbf{l})], \\ p(\mathbf{h} | \mathbf{h}, \mathbf{w})[r(\mathbf{h}, \mathbf{w}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{l} | \mathbf{h}, \mathbf{w})[r(\mathbf{h}, \mathbf{w}, \mathbf{l}) + \gamma v_*(\mathbf{l})] \end{array} \right\} \\
 &= \max \left\{ \begin{array}{l} \alpha[r_{\mathbf{s}} + \gamma v_*(\mathbf{h})] + (1 - \alpha)[r_{\mathbf{s}} + \gamma v_*(\mathbf{l})], \\ 1[r_{\mathbf{w}} + \gamma v_*(\mathbf{h})] + 0[r_{\mathbf{w}} + \gamma v_*(\mathbf{l})] \end{array} \right\} \\
 &= \max \left\{ \begin{array}{l} r_{\mathbf{s}} + \gamma[\alpha v_*(\mathbf{h}) + (1 - \alpha)v_*(\mathbf{l})], \\ r_{\mathbf{w}} + \gamma v_*(\mathbf{h}) \end{array} \right\}.
 \end{aligned}$$

Recycle Robot Problem

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	-

$$\begin{aligned}
 v_*(\mathbf{h}) &= \max \left\{ \begin{array}{l} p(\mathbf{h} | \mathbf{h}, \mathbf{s})[r(\mathbf{h}, \mathbf{s}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{1} | \mathbf{h}, \mathbf{s})[r(\mathbf{h}, \mathbf{s}, \mathbf{1}) + \gamma v_*(\mathbf{1})], \\ p(\mathbf{h} | \mathbf{h}, \mathbf{w})[r(\mathbf{h}, \mathbf{w}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{1} | \mathbf{h}, \mathbf{w})[r(\mathbf{h}, \mathbf{w}, \mathbf{1}) + \gamma v_*(\mathbf{1})] \end{array} \right\} \\
 &= \max \left\{ \begin{array}{l} \alpha[r_{\mathbf{s}} + \gamma v_*(\mathbf{h})] + (1 - \alpha)[r_{\mathbf{s}} + \gamma v_*(\mathbf{1})], \\ 1[r_{\mathbf{w}} + \gamma v_*(\mathbf{h})] + 0[r_{\mathbf{w}} + \gamma v_*(\mathbf{1})] \end{array} \right\} \\
 &= \max \left\{ \begin{array}{l} r_{\mathbf{s}} + \gamma[\alpha v_*(\mathbf{h}) + (1 - \alpha)v_*(\mathbf{1})], \\ r_{\mathbf{w}} + \gamma v_*(\mathbf{h}) \end{array} \right\}.
 \end{aligned}$$

Recycle Robot Problem

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	-

$$\begin{aligned}
 v_*(\mathbf{h}) &= \max \left\{ \begin{array}{l} p(\mathbf{h} | \mathbf{h}, \mathbf{s})[r(\mathbf{h}, \mathbf{s}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{1} | \mathbf{h}, \mathbf{s})[r(\mathbf{h}, \mathbf{s}, \mathbf{1}) + \gamma v_*(\mathbf{1})], \\ p(\mathbf{h} | \mathbf{h}, \mathbf{w})[r(\mathbf{h}, \mathbf{w}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{1} | \mathbf{h}, \mathbf{w})[r(\mathbf{h}, \mathbf{w}, \mathbf{1}) + \gamma v_*(\mathbf{1})] \end{array} \right\} \\
 &= \max \left\{ \begin{array}{l} \alpha[r_{\mathbf{s}} + \gamma v_*(\mathbf{h})] + (1 - \alpha)[r_{\mathbf{s}} + \gamma v_*(\mathbf{1})], \\ 1[r_{\mathbf{w}} + \gamma v_*(\mathbf{h})] + 0[r_{\mathbf{w}} + \gamma v_*(\mathbf{1})] \end{array} \right\} \\
 &= \max \left\{ \begin{array}{l} r_{\mathbf{s}} + \gamma[\alpha v_*(\mathbf{h}) + (1 - \alpha)v_*(\mathbf{1})], \\ r_{\mathbf{w}} + \gamma v_*(\mathbf{h}) \end{array} \right\}.
 \end{aligned}$$

Recycle Robot Problem

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	-

$$\begin{aligned}
 v_*(\mathbf{h}) &= \max \left\{ \begin{array}{l} p(\mathbf{h} | \mathbf{h}, \mathbf{s})[r(\mathbf{h}, \mathbf{s}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{1} | \mathbf{h}, \mathbf{s})[r(\mathbf{h}, \mathbf{s}, \mathbf{1}) + \gamma v_*(\mathbf{1})], \\ p(\mathbf{h} | \mathbf{h}, \mathbf{w})[r(\mathbf{h}, \mathbf{w}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{1} | \mathbf{h}, \mathbf{w})[r(\mathbf{h}, \mathbf{w}, \mathbf{1}) + \gamma v_*(\mathbf{1})] \end{array} \right\} \\
 &= \max \left\{ \begin{array}{l} \alpha[r_{\mathbf{s}} + \gamma v_*(\mathbf{h})] + (1 - \alpha)[r_{\mathbf{s}} + \gamma v_*(\mathbf{1})], \\ 1[r_{\mathbf{w}} + \gamma v_*(\mathbf{h})] + 0[r_{\mathbf{w}} + \gamma v_*(\mathbf{1})] \end{array} \right\} \\
 &= \max \left\{ \begin{array}{l} r_{\mathbf{s}} + \gamma[\alpha v_*(\mathbf{h}) + (1 - \alpha)v_*(\mathbf{1})], \\ r_{\mathbf{w}} + \gamma v_*(\mathbf{h}) \end{array} \right\}.
 \end{aligned}$$

Recycle Robot Problem

$$\begin{aligned} v_*(\mathbf{h}) &= \max \left\{ \begin{array}{l} p(\mathbf{h}|\mathbf{h}, \mathbf{s})[r(\mathbf{h}, \mathbf{s}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{1}|\mathbf{h}, \mathbf{s})[r(\mathbf{h}, \mathbf{s}, \mathbf{1}) + \gamma v_*(\mathbf{1})], \\ p(\mathbf{h}|\mathbf{h}, \mathbf{w})[r(\mathbf{h}, \mathbf{w}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{1}|\mathbf{h}, \mathbf{w})[r(\mathbf{h}, \mathbf{w}, \mathbf{1}) + \gamma v_*(\mathbf{1})] \end{array} \right\} \\ &= \max \left\{ \begin{array}{l} \alpha[r_{\mathbf{s}} + \gamma v_*(\mathbf{h})] + (1 - \alpha)[r_{\mathbf{s}} + \gamma v_*(\mathbf{1})], \\ 1[r_{\mathbf{w}} + \gamma v_*(\mathbf{h})] + 0[r_{\mathbf{w}} + \gamma v_*(\mathbf{1})] \end{array} \right\} \\ &= \max \left\{ \begin{array}{l} r_{\mathbf{s}} + \gamma[\alpha v_*(\mathbf{h}) + (1 - \alpha)v_*(\mathbf{1})], \\ r_{\mathbf{w}} + \gamma v_*(\mathbf{h}) \end{array} \right\}. \end{aligned}$$

Using the same idea for the state **low (l)**, we get:

$$v_*(\mathbf{1}) = \max \left\{ \begin{array}{l} \beta r_{\mathbf{s}} - 3(1 - \beta) + \gamma[(1 - \beta)v_*(\mathbf{h}) + \beta v_*(\mathbf{1})], \\ r_{\mathbf{w}} + \gamma v_*(\mathbf{1}), \\ \gamma v_*(\mathbf{h}) \end{array} \right\}$$

Recycle Robot Problem

Considering the following values and γ equal to 0.9, we now have the nonlinear equation:

s	a	s'	p(s'/s,a)	r(s',a,s)
high	search	high	0.1	-1
high	search	low	0.9	-1
low	search	high	0.6	-3
low	search	low	0.4	-1
high	wait	high	1	-2
low	wait	low	1	-2
low	recharge	high	1	0

$$V(h) = \max \left\{ \begin{array}{l} 0.09V(h) + 0.81V(l) - 1, \\ -2 + 0.9V(h) \end{array} \right\}$$

$$V(l) = \max \left\{ \begin{array}{l} 0.54V(h) + 0.36V(l) - 2.2, \\ -2 + 0.9V(l), \\ 0.9V(h) \end{array} \right\}$$

Recycle Robot Problem

$$V(h) = \max \left\{ \begin{array}{l} 0.09V(h) + 0.81V(l) - 1, \\ -2 + 0.9V(h) \end{array} \right\}$$

$$V(l) = \max \left\{ \begin{array}{l} 0.54V(h) + 0.36V(l) - 2.2, \\ -2 + 0.9V(l), \\ 0.9V(h) \end{array} \right\}$$

To solve it, we need transform this nonlinear equation into linear ones. For that we make a combination of all the equations from the first state and from the second.

For this problem, it will generate 6 linear equations. After solve all of them, the solution will be the higher values.

Instead, we can use iterative methods to compute the values from both AVF and VF with **Dynamic Programming**.

Sutton and Barto — Chapter 4: Dynamic programming

“The term dynamic programming (DP) refers to a collection of algorithms that can be used to compute **optimal policies** given a **perfect model of the environment** as a Markov decision process (MDP).”

“The key idea of DP, and of reinforcement learning generally, is the use of **value functions** to organize and structure the search for good policies.”

“Although DP ideas can be applied to problems with continuous state and action spaces, exact solutions are possible only in special cases.”
It is used mainly with FMDPs.

Policy Evaluation (prediction)

The process of evaluate a policy consist on compute the state-value function obtained by it.

Instead of solve a lot of linear equation, there is a iterative way for policy evaluation.

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π , the policy to be evaluated

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$ arbitrarily, for $s \in \mathcal{S}$, and $V(\text{terminal})$ to 0

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

Policy Evaluation (prediction)

Input π , the policy to be evaluated

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$ arbitrarily, for $s \in \mathcal{S}$, and $V(\text{terminal})$ to 0

Loop: Convergence criterion

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$: Each iteration runs over all states

$v \leftarrow V(s)$ Update the VF with its bellman equation, the input policy and the estimation of the VF for the next state

$$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ See if the values converge

Policy Evaluation - Example

State	Action	Next State	Reward	Prob.
(s ₁)	(a ₁)	(s ₁)	0	1
(s ₁)	(a ₂)	(s ₂)	2	1
(s ₂)	(a ₁)	(T)	1	1
(s ₂)	(a ₂)	(s ₂)	0	1

Policy:

$$\begin{aligned}\pi(a_1 | s_1) &= 0.5 \\ \pi(a_2 | s_1) &= 0.5 \\ \pi(a_1 | s_2) &= 1 \\ \pi(a_2 | s_2) &= 0\end{aligned}$$

Initial

Parameters:

$$\begin{aligned}\gamma &= 0.9 \\ V(T) &= 0 \\ V(s_1) &= 0 \\ V(s_2) &= 0\end{aligned}$$

$$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$$

Algorithm 1st iteration:

$$\begin{aligned}V(s_1) &= 0.5[0+0.9V(s_1)] + \\ &\quad 0.5[2+0.9V(s_2)] \\ V(s_1) &= 0.5[0+0.9 \times 0] + \\ &\quad 0.5[2+0.9 \times 0]\end{aligned}$$

$$V(s_1) = 1$$

$$\begin{aligned}V(s_2) &= 1[1+0.9V(T)] \\ V(s_2) &= 1[1+0.9 \times 0] \\ V(s_2) &= 1\end{aligned}$$

Algorithm 2st iteration:

$$\begin{aligned}V(s_1) &= 0.5[0+0.9V(s_1)] + \\ &\quad 0.5[2+0.9V(s_2)] \\ V(s_1) &= 0.5[0+0.9 \times 1] + \\ &\quad 0.5[2+0.9 \times 1]\end{aligned}$$

$$V(s_1) = 1.9$$

$$\begin{aligned}V(s_2) &= 1[1+0.9V(T)] \\ V(s_2) &= 1[1+0.9 \times 0] \\ V(s_2) &= 1\end{aligned}$$

Policy Evaluation - Example

State	Action	Next State	Reward	Prob.
(s ₁)	(a ₁)	(s ₁)	0	1
(s ₁)	(a ₂)	(s ₂)	2	1
(s ₂)	(a ₁)	(T)	1	1
(s ₂)	(a ₂)	(s ₂)	0	1

Policy:

$$\begin{aligned}\pi(a_1 | s_1) &= 0.5 \\ \pi(a_2 | s_1) &= 0.5 \\ \pi(a_1 | s_2) &= 1 \\ \pi(a_2 | s_2) &= 0\end{aligned}$$

Initial

Parameters:

$$\begin{aligned}\gamma &= 0.9 \\ V(T) &= 0 \\ V(s_1) &= 0 \\ V(s_2) &= 0\end{aligned}$$

$$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$$

Iteration	V(s ₁)	V(s ₂)
0	0.000	0.000
1	1.000	1.000
2	1.900	1.000
3	2.305	1.000
4	2.487	1.000
5	2.569	1.000

Policy Improvement

We first evaluate a policy to then improve it. For it we consider the Action Value Function (AVF)

$$\begin{aligned} q_{\pi}(s, a) &\doteq \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_{\pi}(s')]. \end{aligned}$$

The policy improvement theorem: Let π and π' be any pair of deterministic policies such that, for all $s \in \mathcal{S}$,

$$q_{\pi}(s, \pi'(s)) \geq v_{\pi}(s).$$

Then the policy π' must be as good as π , or better than π .

Obs: $\pi(s)$ is referent to a deterministic policy where only one action as a probability 1, however everything is also used with stochastic policies

The best policy will be the one the choose the actions that given the higher AVF:

$$\pi'(s) \doteq \arg \max_a q_{\pi}(s, a)$$

It also is called as a greedy policy according to the AVF

It intercalates one step of policy evaluation (E) and policy improvement (I) until obtain the optimal policy and estimation of the value functions, given a fully known FMDP.

$$\dots \xrightarrow{I} \pi_* \xrightarrow{E} \mathcal{V}_*,$$

1. Initialization

2. Policy Evaluation

Loop:

$$\Delta \leftarrow 0$$

Loop for each $s \in \mathcal{S}$:

$$v \leftarrow V(s)$$
$$V(s) \leftarrow \sum_{s',r} p(s', r | s, \pi(s)) [r + \gamma V(s')]$$
$$\Delta \leftarrow \max(\Delta, |v - V(s)|)$$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

$$old-action \leftarrow \pi(s)$$
$$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$$

If $old-action \neq \pi(s)$, then $policy-stable \leftarrow false$

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Value Iteration

It runs until convergence the policy evaluation using the best action for each state.

Then runs once the policy improvement to obtain the optimal policy and estimation of the value functions, given a fully known FMDP.

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$   
| Loop for each  $s \in \mathcal{S}$ :  
|    $v \leftarrow V(s)$   
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$   
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$   
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that
$$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$$

External Links

https://cs.stanford.edu/people/karpathy/reinforcejs/gridworld_dp.html

https://colab.research.google.com/drive/1rzw3LIKx02u_JLcW-unjXm5rBebXevj?authuser=1#scrollTo=KJ_k10iNhACj